

JPR Assignment 1

1. Explain Java tokens

Ans. In Java, tokens are the smallest individual units of a program that the compiler can recognize. Just as words, punctuation, and symbols make up a sentence in English, tokens are the building blocks of Java source code.

When you compile a program, the compiler breaks the code down into these tokens during a process called lexical analysis.

Types of Java Tokens

Java recognizes five primary categories of tokens:

1. Keywords

Keywords are reserved words that have a predefined meaning in Java. You cannot use these as identifiers (names for variables or classes).

- Examples: public, class, static, void, int, if, else, while.

2. Identifiers

Identifiers are the names given by the programmer to various elements like variables, methods, classes, and interfaces.

- Rules: They must start with a letter, \$, or _. They cannot start with a digit or be a keyword.
- Examples: main, system, myVariable, calculateSum.

3. Literals

Literals represent constant values stored in variables. They are the "raw data" in your code.

- Integer Literals: 10, 100
- Floating-point Literals: 10.5, 3.14
- Character Literals: 'A', '9'
- String Literals: "Hello Java"
- Boolean Literals: true, false

4. Operators

Operators are symbols used to perform operations on variables and values.

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, >, <
- Logical: &&, ||, !
- Assignment: =, +=

5. Separators (Punctuators)

Separators are symbols used to organize and structure the code.

- Parentheses (): Used for method arguments and expressions.
- Braces {}: Used to define blocks of code (classes/methods).
- Brackets []: Used for arrays.
- Semicolon ;: Used to terminate statements.
- Comma ,: Used to separate variables or parameters.

Example Breakdown

Consider this simple line of code: `int count = 50;`

The compiler sees this as four distinct tokens:

1. `int` — Keyword
2. `count` — Identifier
3. `=` — Operator
4. `50` — Literal
5. `;` — Separator

2. Explain String and methods of String

Ans. In Java, a String is an object that represents a sequence of characters. Unlike primitive data types (like `int` or `char`), String is a class located in the `java.lang` package.

One of the most important things to remember about Strings is that they are immutable. This means once a String object is created, its value cannot be changed. If you try to modify it, Java actually creates a brand new String object in memory.

Common String Methods

The String class provides a variety of built-in methods to manipulate and analyze text. Here are the most frequently used ones:

1. Basic Information

- `length()`: Returns the number of characters in the string.
 - `"Hello".length()` returns 5.
- `charAt(int index)`: Returns the character at a specific index (starting from 0).
 - `"Java".charAt(2)` returns 'v'.

2. Comparison

- `equals(Object obj)`: Compares the content of two strings for equality.
- `equalsIgnoreCase(String str)`: Compares two strings, ignoring whether letters are uppercase or lowercase.
- `compareTo(String str)`: Compares strings lexicographically (dictionary order).

3. Transformation (Returns a New String)

- `toUpperCase()` / `toLowerCase()`: Converts the entire string to upper or lower case.
- `trim()`: Removes leading and trailing whitespace.
- `replace(char old, char new)`: Replaces all occurrences of a specific character.
- `substring(int begin, int end)`: Extracts a portion of the string from the begin index up to (but not including) the end index.

4. Searching

- `indexOf(String str)`: Returns the index of the first occurrence of the specified substring.
- `contains(CharSequence s)`: Returns true if the string contains the specified sequence of characters.
- `startsWith(String prefix)` / `endsWith(String suffix)`: Checks if the string begins or ends with a specific pattern.

3. WAP for if-else-if ladder

Ans. import java.util.Scanner;
public class GradeSystem {
 public static void main(String[] args) {
 Scanner input = new Scanner(System.in);
 System.out.print("Enter your score (0-100): ");
 int score = input.nextInt();
 // The if-else-if ladder starts here
 if (score >= 90) {
 System.out.println("Grade: A+");
 }
 else if (score >= 80) {
 System.out.println("Grade: A");
 }
 else if (score >= 70) {
 System.out.println("Grade: B");
 }
 else if (score >= 60) {
 System.out.println("Grade: C");
 }
 else if (score >= 50) {
 System.out.println("Grade: D");
 }
 else {
 System.out.println("Grade: F (Fail)");
 }

 input.close();
 }
}

4. WAP to implement String buffer method

Ans. public class StringBufferDemo {
 public static void main(String[] args) {
 // 1. Initialize StringBuffer
 StringBuffer sb = new StringBuffer("Hello");

 // 2. append() - Adds text to the end of the existing string
 sb.append(" Java");
 System.out.println("After append: " + sb); // Output: Hello Java

 // 3. insert() - Inserts text at a specific index
 sb.insert(6, "World ");
 }
}

```

System.out.println("After insert: " + sb); // Output: Hello World Java

// 4. replace() - Replaces characters from start index to end index
sb.replace(6, 11, "Python");
System.out.println("After replace: " + sb); // Output: Hello Python Java

// 5. delete() - Removes characters from start index to end index
sb.delete(5, 12);
System.out.println("After delete: " + sb); // Output: Hello Java

// 6. reverse() - Reverses the sequence of characters
sb.reverse();
System.out.println("After reverse: " + sb); // Output: ava lloH

// 7. capacity() - Returns the total allocated memory (default is length + 16)
System.out.println("Current Capacity: " + sb.capacity());
}
}

```

5. Define constructor. Explain types of constructor with code

Ans. In Java, a constructor is a special block of code that is used to initialize a newly created object. It is called automatically when an instance of a class is created using the new keyword.

Key Characteristics:

- Name: It must have the exact same name as the class.
- Return Type: It does not have a return type (not even void).
- Purpose: To set initial values for object attributes.

Types of Constructors

1. Default Constructor

If you do not write any constructor in your class, the Java compiler automatically inserts a "default" constructor for you. It initializes member variables with default values (e.g., 0 for integers, null for objects).

2. No-Argument Constructor

This is a constructor defined by the programmer that has no parameters. It is often used to provide custom default values to an object.

```

class Dog {
    String breed;
    // No-Argument Constructor
    Dog() {
        breed = "Unknown";
        System.out.println("A dog object is created.");
    }
}

```

3. Parameterized Constructor

A constructor that accepts arguments is called a parameterized constructor. This allows you to assign different values to different objects at the time of their creation.

```
class Student {  
    String name;  
    int rollNo;  
  
    // Parameterized Constructor  
    Student(String n, int r) {  
        name = n;  
        rollNo = r;  
    }  
  
    void display() {  
        System.out.println("Name: " + name + ", Roll No: " + rollNo);  
    }  
}
```